# Estrategia y arquitectura: Fase A

Manuel Zuñiga

Edwin Bolaños

Paula Ayala

Dayra Quenguan

Kevin Rodriguez

Pruebas de software e implementación – Tecnología en desarrollo de software

Institucion universitaria del putumayo

Mocoa - Putumayo

## Introducción

Este documento presenta la estrategia de pruebas para CineFlex, una plataforma web simulada de venta de entradas de cine. El propósito es definir decisiones de integración, requisitos verificables, criterios de regresión y una comparación contextualizada de Selenium WebDriver y Cypress.

La estrategia se plantea para un entorno de pruebas (mocks) sin la necesidad de usar datos reales

## Contexto y alcance

El alcance de esta estrategia corresponde a la Fase A del taller:

1. Pruebas de integración mediante stubs y drivers;
2. requisitos funcionales y no funcionales junto con la selección de pruebas de regresión; y
3. comparación técnica entre Selenium WebDriver y Cypress.

## Pruebas de integración

### Enfoque usado: Top-Down con Stub

Se recomienda un enfoque Top-Down para comenzar desde el flujo de negocio visible en la interfaz y avanzar hacia los componentes inferiores, el flujo que interesa validar comienza cuando el usuario selecciona una película y un asiento, introduce los datos de pago y confirma la reserva. Como la pasarela bancaria externa aún no está disponible, un Stub reemplazará temporalmente esa dependencia.

Esto permite comprobar pronto la coordinación entre la interfaz, el endpoint de reserva y la lógica que interpreta la respuesta del servicio de pagos mientras que el stub representa la respuesta del componente externo

## Diseño del Stub de pagos

El Stub será un módulo de prueba controlado que reciba una solicitud de autorización con un identificador de transacción, importe y datos de tarjeta ficticios. Nunca debe utilizarse información bancaria real. Para hacer los resultados reproducibles, el Stub devolverá respuestas predeterminadas según el escenario solicitado.

| Escenario | Respuesta simulada | Comportamiento esperado de CineFlex |
|---|---|---|
| **Pago aprobado** | HTTP 200; estado APPROVED; transactionId ficticio. | La API registra el resultado simulado y responde con confirmación de reserva y ticket de prueba. |
| **Pago rechazado** | HTTP 402; estado DECLINED; código de rechazo. | La API informa que el pago no fue aprobado y no confirma la compra. |
| **Error del proveedor** | HTTP 503 o timeout controlado. | La API devuelve un error controlado, no expone detalles internos y no genera una confirmación falsa. |
| **Respuesta inválida** | JSON incompleto o estado desconocido. | La API trata la respuesta como error de integración y evita marcar el pago como aprobado. |

## Implementación y ejecución

Se recomienda definir una interfaz interna para el proveedor de pagos, por ejemplo paymentGateway.authorize(paymentData), y configurar una implementación simulada durante las pruebas. El endpoint POST /reserva llamará a esa interfaz en lugar de depender directamente de un servicio externo. La selección del Stub se hará mediante configuración del entorno de pruebas

- Prepara datos de prueba: película, función, asiento disponible e importe ficticio.
- Enviar una solicitud de reserva al backend y comprobar que se invoque el Stub.
- Repetir el caso con respuestas aprobada, rechazada y error del proveedor.
- Confirmar que una respuesta rechazada o fallida nunca produzca un ticket confirmado.
- Registrar la petición y el resultado sin escribir números de tarjeta, secretos ni datos sensibles en los logs.

## Pruebas de sistema: requisitos funcionales y no funcionales

### Requisitos funcionales

- ✓ **RF-01. Consulta de cartelera.** El sistema debe permitir que un usuario autenticado consulte la cartelera y recibir una respuesta JSON válida con la información de las películas disponibles. Criterio de aceptación: GET /cartelera devuelve HTTP 200 con token válido y una estructura de datos que cumple el contrato definido.
- ✓ **RF-02. Reserva y rechazo de pago.** El sistema debe procesar la solicitud de reserva y reflejar el resultado de autorización del pago simulado. Criterio de aceptación: si el Stub aprueba, se devuelve una confirmación de prueba; si rechaza, la API devuelve un error de negocio (por ejemplo, HTTP 400 o 402) y no confirma la compra.

### Requisitos no funcionales

- ✓ **RNF-01. Rendimiento de cartelera.** El endpoint GET /cartelera debe responder en menos de 300 ms bajo las condiciones controladas de la prueba de API. Se medirá el tiempo por petición en Postman/Newman; el resultado se registrará como evidencia y no se asumirá sin ejecutar la prueba.
- ✓ **RNF-02. Manejo seguro de errores.** Los errores de autenticación, reserva o proveedor no deben revelar secretos, trazas internas ni datos de tarjeta. Criterio de aceptación: las respuestas contienen un mensaje controlado y los registros omiten datos sensibles.

## Estrategia de pruebas de regresión

**A. Cambio visual en el perfil**

Si el cambio se limita al color de un botón secundario de la vista de perfil y no modifica código compartido, navegación, autenticación ni lógica de negocio, no es necesario ejecutar de inmediato toda la suite de pruebas de la pasarela de pagos

La revisión debe basarse en ejecutar primero pruebas de la vista de perfil y una comprobación breve de navegación; después ampliar la regresión si el cambio afecta estilos globales, componentes compartidos, scripts, rutas o dependencias utilizadas por el flujo de compra.

**B. No se omite la regresión por conveniencia:**

se limita inicialmente a los casos con relación técnica o funcional con el cambio, la compra y el pago deben incluirse en una regresión más amplia antes de una entrega relevante, después de cambios en reserva/autenticación o cuando la evidencia indique riesgo de efectos colaterales.

## Niveles propuestos

| Nivel | Cuándo se ejecuta | Casos incluidos |
|---|---|---|
| Regresión focalizada | Tras un cambio pequeño y localizado. | Pruebas de la funcionalidad modificada, renderizado del perfil y navegación relacionada. |
| Regresión de integración | Si cambia API, autenticación, contrato o componente compartido. | Login, acceso autenticado a cartelera, reserva y manejo de errores de pago. |
| Regresión completa | Antes de una versión importante, tras cambios de alto riesgo o al integrar varias ramas. | Suite de API, flujos UI críticos, compra de extremo a extremo, cierre de sesión y navegación segura. |

## Comparativa técnica: Selenium WebDriver y Cypress

| Criterio | Selenium WebDriver | Cypress | Consideraciones para CineFlex |
|---|---|---|---|
| **Arquitectura** | Maneja el navegador mediante WebDriver y soporta varios lenguajes. | Corre las pruebas directo dentro del navegador e incluye herramientas para depurar. | Selenium se integra sin problema con el entorno de Node.js que estamos usando; Cypress sirve bastante para avanzar rápido en la interfaz si nos adaptamos a su forma de trabajo. |
| **Navegadores y entornos** | Funciona con casi cualquier navegador y se presta para ejecuciones remotas o en servidores. | La compatibilidad con navegadores depende del release y la configuración que se le haga. | Si la prioridad es probar la plataforma en diferentes navegadores o correr en red, Selenium da más libertad. |
| **Depuración** | Depende de revisar logs, tomar capturas de pantalla y manejar tiempos de espera. | Tiene una interfaz interactiva donde se ve cada comando y el estado de la app al instante. | Para detectar fallos en la cartelera o en los botones durante el desarrollo, Cypress permite encontrar errores mucho más rápido. |
| **Organización del código** | Facilita estructurar el proyecto con Page Object Model (POM), separando la lógica de la página de las validaciones. | Permite crear comandos personalizados y reusar componentes, siguiendo las convenciones que se definan. | La idea es estructurar la app con POM en Selenium: dejar los selectores y las interacciones en las clases de cada vista (LoginPage, CarteleraPage, ReservaPage) y mover las validaciones al archivo de pruebas (test_suite.js). |
| **Caso de uso principal** | Pruebas de compatibilidad en varios navegadores, integración con infraestructura existente y arquitectura modular. | Pruebas de interfaz y flujos de usuario con retroalimentación inmediata mientras se programa. | Lo mejor es usar Selenium como el motor base del proyecto por su modularidad y dejar Cypress como una alternativa si necesitamos agilizar pruebas visuales más adelante. |

## Criterios de entrada y salida

### A. Criterios de entrada

El backend mock y el frontend pueden iniciarse siguiendo el README del repositorio

Las variables de entorno de prueba están configuradas y no contienen secretos reales.

Los datos de prueba son conocidos y reproducibles

### B. Criterios de salida

- Los casos planificados se ejecutaron y sus resultados quedaron registrados
- No hay defectos críticos abiertos en los flujos incluidos en el alcance
- Los rechazos y errores no generan confirmaciones de compra falsas
- El resultado de rendimiento se respalda con evidencia de ejecución real
- Las limitaciones del mock y los casos pendientes se reportan con transparencia

### C. Riesgos y limitaciones

- **Pasarela bancaria no disponible.** El Stub permite probar el contrato y los comportamientos esperados, pero no reemplaza la validación real del proveedor.
- **Backend de demostración.** Los resultados solo aplican al mock y no prueban que la plataforma esté lista para producción.
- **Cobertura incompleta.** El repositorio identifica como pendientes casos adicionales de API y UI
- **Pruebas de carga.** Los valores de throughput, porcentaje de error y latencia deben obtenerse ejecutando JMeter